

```
-- =====
-- FlipCart Products Database — Week 3 Assessment Dataset
-- The Unlox Academy · DA/DS Track
-- =====
-- HOW TO RUN:
-- 1. Open MySQL Workbench
-- 2. File > Open SQL Script... > select this file
-- 3. Press Ctrl+Shift+Enter to run all
-- 4. Refresh Schemas panel — you should see 'flipcart' with 'products' table
-- 5. Verify with: SELECT COUNT(*) FROM flipcart.products; -- should return 30
-- =====
```

```
CREATE DATABASE IF NOT EXISTS flipcart;
USE flipcart;
```

```
DROP TABLE IF EXISTS products;
```

```
CREATE TABLE products (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(80),
  category VARCHAR(30),
  brand VARCHAR(30),
  price DECIMAL(10, 2),
  stock_quantity INT,
  is_active BOOLEAN,
  launched_date DATE,
  avg_rating DECIMAL(3, 2),
  discount_pct INT
);
```

```
INSERT INTO products VALUES
```

```
-- Electronics (8 products)
(1, 'Sony Bravia 55 TV', 'Electronics', 'Sony', 65000.00, 12, TRUE, '2022-05-10', 4.50, 15),
(2, 'Apple iPhone 15', 'Electronics', 'Apple', 79999.00, 25, TRUE, '2023-09-22', 4.80, 5),
(3, 'boAt Airdopes 141', 'Electronics', 'boAt', 1499.00, 200, TRUE, '2022-11-15', 4.20, 30),
(4, 'Xiaomi Redmi Buds 4', 'Electronics', 'Xiaomi', 799.00, 350, TRUE, '2023-03-08', 4.00, 20),
(5, 'Samsung Galaxy Watch', 'Electronics', 'Samsung', 22999.00, 45, TRUE, '2023-06-15', 4.40, 10),
(6, 'OnePlus Nord Buds 2', 'Electronics', 'OnePlus', 2199.00, 180, TRUE, '2024-01-10', NULL, 25),
(7, 'JBL Flip 6 Speaker', 'Electronics', 'JBL', 4999.00, 0, TRUE, '2022-07-20', 4.30, 15),
(8, 'Nokia 105', 'Electronics', 'Nokia', 1499.00, 500, FALSE, '2019-01-15', 3.80, 0),
-- Apparel (6 products)
(9, 'Nike Air Max Shoes', 'Apparel', 'Nike', 8999.00, 65, TRUE, '2023-02-20', 4.60, 20),
(10, 'Levis 511 Slim Jeans', 'Apparel', 'Levis', 2999.00, 120, TRUE, '2022-08-15', 4.30, 30),
(11, 'Puma Essential T-Shirt', 'Apparel', 'Puma', 1299.00, 250, TRUE, '2023-05-10', 4.10, 40),
(12, 'Van Heusen Formal Shirt', 'Apparel', 'Van Heusen', 1899.00, 90, TRUE, '2022-09-25', 4.20, 25),
(13, 'Fabindia Cotton Kurti', 'Apparel', 'Fabindia', 2499.00, 75, TRUE, '2024-02-01', NULL, 10),
(14, 'Adidas Track Pants', 'Apparel', 'Adidas', 3499.00, 130, TRUE, '2023-01-15', 4.50, 15),
-- Home (5 products)
```

```

(15, 'Prestige Pressure Cooker', 'Home', 'Prestige', 1999.00, 85, TRUE, '2022-06-10',
4.40, 20),
(16, 'Milton Water Bottle 1L', 'Home', 'Milton', 449.00, 400, TRUE, '2023-03-15', 4.00,
25),
(17, 'Godrej Bedsheet Set', 'Home', 'Godrej', 1499.00, 150, TRUE, '2023-08-20', 4.20,
30),
(18, 'IKEA Study Table', 'Home', 'IKEA', 5999.00, 40, TRUE, '2024-01-20', NULL,
0),
(19, 'Havells Ceiling Fan', 'Home', 'Havells', 3299.00, 60, TRUE, '2022-04-15', 4.30,
15),
-- Books (4 products)
(20, 'The Silent Patient', 'Books', 'Alex Michaelides', 399.00, 200, TRUE, '2020-07-15',
4.50, 20),
(21, 'Atomic Habits', 'Books', 'James Clear', 349.00, 350, TRUE, '2019-10-16',
4.70, 25),
(22, 'Rich Dad Poor Dad', 'Books', 'Robert Kiyosaki', 299.00, 180, TRUE, '2017-04-01',
4.40, 30),
(23, 'Sapiens', 'Books', 'Yuval Harari', 499.00, 90, TRUE, '2018-11-15', 4.60,
15),
-- Beauty (4 products)
(24, 'Nykaa Matte Lipstick', 'Beauty', 'Nykaa', 599.00, 320, TRUE, '2023-04-10', 4.20,
20),
(25, 'Lakme Eye Liner', 'Beauty', 'Lakme', 299.00, 500, TRUE, '2022-05-20', 4.10,
25),
(26, 'Mamaearth Face Wash', 'Beauty', 'Mamaearth', 249.00, 400, TRUE, '2023-07-15',
4.30, 30),
(27, 'WOW Skin Vitamin C Serum', 'Beauty', 'WOW', 899.00, 0, TRUE, '2023-11-10',
4.50, 40),
-- Sports (3 products)
(28, 'Cosco Cricket Bat', 'Sports', 'Cosco', 1299.00, 45, TRUE, '2022-10-15', 4.00,
10),
(29, 'Yonex Badminton Racket', 'Sports', 'Yonex', 2499.00, 80, TRUE, '2023-06-20', 4.40,
15),
(30, 'Nivia Football Size 5', 'Sports', 'Nivia', 899.00, 120, TRUE, '2023-05-05', 4.20, 25);

```

```

-- Verify the data loaded
SELECT COUNT(*) AS total_products FROM products;
SELECT category, COUNT(*) AS count FROM products GROUP BY category ORDER BY count
DESC;

```

----- SECTION A - THEORY -----

```

-- A1. C --
-- A2. B --
-- A3. B --
-- A4. C --
-- A5. B --
-- A6. C --
-- A7. B --
-- A8. C --

```

----- SECTION B - OUTPUT PREDICTION -----

```

-- B1. What does this query return? --
SELECT COUNT(*) FROM products WHERE category ='Electronics';
-- Your answer:
-- This query returns the total number of products in the Electronics category.

-- B2. How many rows does this query return?
SELECT * FROM products WHERE price BETWEEN 1000 AND 3000;
-- Your answer:11

```

-- B3. What is the output of this query?

```
SELECT product_name FROM products
WHERE category =
'Books' AND price < 400
ORDER BY price DESC
LIMIT 1;
```

-- Your answer: This query returns the name of the most expensive book that costs less than ₹400. It shows only one product name .

-- B4. How many rows does this query return?

```
SELECT * FROM products WHERE avg_rating IS NULL;
```

-- Your answer: 3

-- B5. What single value does this query return?

```
SELECT MAX(price) FROM products WHERE category = 'Books';
```

-- Your answer: This query returns the highest price of the book among the books category .

-- B6. What does this query return?

```
SELECT category, COUNT(*) FROM products
WHERE is_active = TRUE
GROUP BY category
HAVING COUNT(*) > 4;
```

-- Your answer: This query shows each category that has more than 4 active products.

-- B7. What does the output look like?

```
SELECT product_name,
       CASE
           WHEN price < 500 THEN 'Budget'
           WHEN price < 5000 THEN 'Mid'
           ELSE 'Premium'
       END AS tier
FROM products WHERE category = 'Beauty';
```

-- Your answer: (COPY-PASTED THE OUTPUT FOR GOOD VIEWING)

```
-- | product_name | tier |
-- | Nykaa Matte Lipstick | Mid |
-- | Lakme Eye Liner | Budget |
-- | Mamaearth Face Wash | Budget |
-- | WOW Skin Vitamin C Serum | Mid |
```

-- B8. What does this query return?

```
SELECT product_name, COALESCE(avg_rating, 0) AS rating
FROM products
WHERE stock_quantity = 0;
```

-- Your answer: This query returns the names and ratings of all products that are out of stock.

-- ----- SECTION C - APPLIED SQL -----

-- C1 - Basic SELECT + WHERE + ORDER BY

-- C1. Write a query to display all products with all columns.

-- Your query:

```
Select *from products;
```

-- C2. Write a query that shows the product_name and price of all Books.

-- Your query:

```
Select product_name, price
from products
where category = 'Books';
```

-- C3. Write a query to list all products priced above ₹10,000, sorted from highest price to lowest.
 -- Your query:
 select *from products where price>10000 order by price desc;

-- C4. Write a query to display the top 5 most expensive products in the Electronics category(show product name and price).
 -- Your query:
 select product_name, price from products where category = 'Electronics' order by price desc limit 5;

-- C2 - Filtering Variations (IN, BETWEEN, LIKE)

-- C5. Write a query to list all products in either Electronics or Apparel category. Use the IN operator.
 -- Your query:
 select *from products where category = 'Electronics' or category = 'Apparel';

-- C6. Write a query to find all products with a price between ₹500 and ₹2,000 (inclusive).
 -- Your query:
 select *from products where price between 500 and 2000;

-- C7. Write a query to find all products whose product_name contains the word 'Watch'.
 -- Your query:
 select *from products where product_name like '%watch%';

-- C8. Write a query to find all products whose brand starts with the letter 'S'.
 -- Your query:
 select
 *from products where brand like 's%';

-- C3 - DISTINCT + Aggregate Functions

-- C9. Write a query to list all the unique categories in the products table.
 -- Your query:
 select distinct category from products;

-- C10. Write a query to find the total number of products in the entire catalogue.
 -- Your query:
 select count(*) as total_products from products;

-- C11. Write a query to find the average price of all Books.
 -- Your query:
 select avg(price) as avg_price from products where category='Books';

-- C12. Write a single query that returns the maximum and minimum price across all products.
 -- Your query:
 select max(price) as max_price,min(price) as min_price from products;

-- C4 - GROUP BY

-- C13. Write a query to count how many products are in each category.
 -- Your query:
 select category, count(*) as product_count from products group by category;

-- C14. Write a query to show the total stock quantity for each category.
 -- Your query:
 select category, sum(stock_quantity) as total_stock from products group by category;

-- C15. Write a query to show the average price per category, sorted from highest average to lowest.

-- Your query:

```
select category, avg(price) as avg_price from products
group by category order by avg_price desc;
```

-- C16. Write a query to show the count of products and the average price for each brand, showing only brands that have more than 1 product.

-- Your query:

```
select brand, count(*) as product_count , avg(price) as avg_price
from products group by brand having count(*)>1;
```

-- C5 - HAVING + LIMIT

-- C17. Write a query to find categories that have more than 4 active products.

-- Your query:

```
select category from products where is_active = 1 group by category having count(*) > 4;
```

-- C18. Write a query to find the 3 most expensive products in the entire catalogue.

-- Your query:

```
select product_name, price from products order by price desc limit 3;
```

-- C19. Write a query to find all categories where the average price is above ₹2,000.

-- Your query:

```
select category from products group by category having avg(price) > 2000;
```

-- C6 - NULL Handling

-- C20. Write a query to list all products where the avg_rating is missing.

-- Your query:

```
select * from products where avg_rating is null;
```

-- C21. Write a query to show product_name and rating for all products. If avg_rating is NULL, display the text 'New Launch' instead.

-- Your query:

```
select product_name, if null (avg_rating, 'New Launch') as rating from products;
```

-- C7 - CASE WHEN

-- C22. Write a query to display each product with a price_tier column classified as 'Budget' if price < ₹1,000, 'Mid' if price < ₹10,000, or 'Premium' if price >= ₹10,000.

-- Your query:

```
select product_name, price,
       case
         when price < 1000 then 'Budget'
         when price < 10000 then 'Mid'
         else 'Premium'
       end as price_tier
from products;
```

-- C23. For each category, write a query that shows the total product count and the count of 'Premium' products (price >= ₹10,000). Use SUM(CASE WHEN...).

-- Your query:

```
select category , count(*) as total_products , sum(case when price >= 10000 then 1
else 0 end) as premium_products from products group by category;
```

-- C24. Write a comprehensive query showing for each category (only categories with at least 3

-- products): total product count, count of active products, average price, and a category_tier

-- column ('Cheap' if avg < ₹1500, 'Standard' if avg < ₹10000, 'Luxury' if avg >= ₹10000). Sort by average price descending.

```
-- Your query:
select category,count(*) as total_products,
sum(is_active) as active_products,
avg(price) as average_price,
    case
        when avg(price) < 1500 then 'Cheap'
        when avg(price) < 10000 then 'Standard'
        else 'Luxury'
    end as category_tier
from products group by category
having count(*) >= 3
order by avg(price) desc;
```

— not able to convert sql file to pdf so copied the whole text and made pdf